

5.1 ADALINE (Adaptive Linear Neuron) (1960) & Hoff (1960)

Part (a): ADALINE Model and Learning Algorithm

Overview

ADALINE (**Adaptive Linear Neuron**) was introduced by Widrow & Hoff (1960) as an improvement over the Perceptron. The key distinction is that ADALINE computes the error on the **raw linear output** (before thresholding), enabling the use of a continuous, differentiable cost function and a stable gradient-based learning rule.

Model

Given input $\mathbf{x} \in \mathbb{R}^d$ and label $y \in \{-1, +1\}$, augment the input as $\tilde{\mathbf{x}} = [x_1, \dots, x_d, 1]^\top$ and $\tilde{\mathbf{w}} = [w_1, \dots, w_d, b]^\top$. The net input is:

$$\text{net} = \mathbf{w}^\top \mathbf{x} + b = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}$$

The predicted class label is:

$$\hat{y} = \text{sign}(\text{net})$$

During training, the error is measured on **net**, not on $\hat{y}$. This is the defining feature of ADALINE.

Cost Function

ADALINE minimizes the **Mean Squared Error (MSE)**:

$$J(\tilde{\mathbf{w}}) = \frac{1}{2} \sum_{i=1}^n (y^{(i)} - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}^{(i)})^2$$

This is a **convex quadratic** function in $\tilde{\mathbf{w}}$, meaning it has no local minima — only a single global minimum.

Learning Algorithm (Widrow–Hoff / Delta Rule)

For a single training example $(\tilde{\mathbf{x}}^{(i)}, y^{(i)})$, the gradient of the loss with respect to $\tilde{\mathbf{w}}$ is $-e^{(i)}\tilde{\mathbf{x}}^{(i)}$, yielding the update rule:

$$\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta \cdot e^{(i)} \cdot \tilde{\mathbf{x}}^{(i)}, \quad e^{(i)} = y^{(i)} - \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}^{(i)}$$

Algorithm:

ADALINE Learning Algorithm

Input: Training set $\{(\tilde{\mathbf{x}}^{(i)}, y^{(i)})\}_{i=1}^n$, learning rate η

1. Initialize $\tilde{\mathbf{w}} \leftarrow \mathbf{0}$
 2. **For** each training example $(\tilde{\mathbf{x}}^{(i)}, y^{(i)})$:
 - a. Compute net input: $\text{net}^{(i)} = \tilde{\mathbf{w}}^\top \tilde{\mathbf{x}}^{(i)}$
 - b. Compute error: $e^{(i)} = y^{(i)} - \text{net}^{(i)}$
 - c. Update weights: $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + \eta \cdot e^{(i)} \cdot \tilde{\mathbf{x}}^{(i)}$
 3. Repeat until convergence
 4. **Predict:** $\hat{y} = \text{sign}(\tilde{\mathbf{w}}^\top \tilde{\mathbf{x}})$
-

Part (b): Convergence to the Global Least-Squares Solution

Step 1: Matrix Formulation

Let $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$ be the design matrix (rows are $\tilde{\mathbf{x}}^{(i)\top}$) and $\mathbf{y} \in \mathbb{R}^n$ the label vector. The cost function becomes:

$$J(\tilde{\mathbf{w}}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\tilde{\mathbf{w}}\|^2$$

Step 2: Convexity — Hessian is Positive Semi-Definite

The Hessian of J with respect to $\tilde{\mathbf{w}}$ is:

$$\nabla^2 J = \mathbf{X}^\top \mathbf{X}$$

For any $\mathbf{v} \in \mathbb{R}^{d+1}$:

$$\mathbf{v}^\top (\mathbf{X}^\top \mathbf{X}) \mathbf{v} = \|\mathbf{X}\mathbf{v}\|^2 \geq 0$$

Therefore $\mathbf{X}^\top \mathbf{X} \succeq 0$, confirming J is **globally convex**. Gradient descent on a convex function is guaranteed to reach the global minimum.

Step 3: Unique Global Minimum via Normal Equations

Setting $\nabla_{\tilde{\mathbf{w}}} J = 0$:

$$-\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\tilde{\mathbf{w}}) = \mathbf{0} \implies \mathbf{X}^\top \mathbf{X} \tilde{\mathbf{w}}^* = \mathbf{X}^\top \mathbf{y}$$

$$\boxed{\tilde{\mathbf{w}}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} = \mathbf{X}^+ \mathbf{y}}$$

This is the **global least-squares solution**, which always exists and is unique when $\mathbf{X}$ has full column rank.

Step 4: Convergence Condition

The online weight updates converge to $\tilde{\mathbf{w}}^*$ provided the learning rate satisfies:

$$0 < \eta < \frac{2}{\lambda_{\max}(\mathbf{X}^\top \mathbf{X})}$$

where $\lambda_{\max}$ is the largest eigenvalue of $\mathbf{X}^\top \mathbf{X}$. Under this condition:

$$\mathbb{E}[\tilde{\mathbf{w}}(t)] \longrightarrow \tilde{\mathbf{w}}^* \quad \text{as } t \rightarrow \infty$$

Step 5: Independence from Data Separability

	Perceptron	ADALINE
Error signal	On $\hat{y}$ (after threshold)	On net (before threshold)
Cost function	No explicit objective	Convex quadratic MSE
Convergence	Only if linearly separable	Always guaranteed
Solution quality	Any separating hyperplane	Globally optimal least-squares

The Perceptron update rule is only guaranteed to converge when the data is linearly separable; on non-separable data it cycles indefinitely. ADALINE has no such dependency. Since $J(\tilde{\mathbf{w}})$ is convex and bounded below by zero, gradient descent must converge to $\tilde{\mathbf{w}}^*$ regardless of whether a separating hyperplane exists.